BLE Mesh Communication using ESP Devices

Tejas Raichura

Electronics and Communication Engineering

NIRMA UNIVERSITY

Ahmedabad, Gujarat, India

Abstract

This report explains how I built a two-way smart network using Bluetooth and the cloud. Usually, smart devices (like plugs or sensors) all connect to Wi-Fi individually, which uses too much power and slows down the router. To fix this, I used a Bluetooth Mesh network built with ESP32 microcontrollers. In my setup, only one central device (the Gateway) connects to the internet. A Sensor node sends temperature data to the Gateway via Bluetooth, which then uploads it to the Anedya Cloud. At the same time, the Gateway checks the cloud for user commands and tells a Relay node to turn a physical switch on or off.

I. Problem Statement

The objective of this project is to design and implement a Bluetooth Low Energy (BLE) Mesh-based communication system using multiple ESP devices.

The system must fulfill the following core requirements:

- Read temperature and humidity data from a physical DHT sensor.
- Control a hardware relay.
- Transmit all collected device data securely to the Anedya Cloud.
- Enable the relay to be controlled remotely through commands received directly from the Anedya Cloud.

To achieve this, the network must use a decentralized communication architecture. One ESP32 device must operate as a ”Gateway Node” with active internet connectivity via Wi-Fi to bridge the BLE Mesh network with the cloud. The remaining ESP32 nodes will function as sensor and actuator endpoints, collecting data and sharing it across the BLE Mesh. These nodes must communicate reliably using multi-hop messaging protocols, ensuring that data can bounce from device to device to provide wide coverage and fault tolerance across the network.

II. Project Overview

To solve the problem statement, I built a highly efficient system that connects a local, low-power Bluetooth network to the global internet using ESP32 microcontrollers. The ESP32 was specifically chosen for this task because it features both Wi-Fi and Bluetooth hardware built directly into the same chip, making it the perfect physical bridge between the two distinct technologies.

In a standard smart home or industrial factory, if there are 50 smart devices, all 50 devices will independently try to connect to the main Wi-Fi router. This traditional approach uses a massive amount of electricity, requires expensive, high-bandwidth routers, and can easily cause the local network to crash under heavy traffic. I solved this bottleneck by using a ”Bluetooth Mesh” architecture. Instead of every device shouting directly at the router, devices in a mesh network talk to each other like a bucket brigade.

If a sensor is too far away from the main internet connection, it simply passes its data packet to a neighboring device, which passes it to the next neighbor, until the message successfully reaches its destination. This multi-hop routing technology also means that if one intermediate device completely breaks or loses power, the network automatically heals itself and finds a new alternative path to send the data. This decentralized approach not only reduces the load on the central Wi-Fi router but also significantly lowers the overall power consumption of the individual nodes.

To demonstrate this robust system, I set up three distinct ESP32 microcontrollers:

1) **Sensor Node (The Endpoint):** Reads real-time temperature and humidity from a physical DHT environment sensor and broadcasts it over the Bluetooth network.
2) **Relay Node (The Actuator):** Waits silently for a Bluetooth command signal to safely turn a physical high-voltage power switch (a relay) on or off.
3) **Gateway Node (The Bridge):** The ”boss” of the local network. It is the only device actively connected to Wi-Fi. It catches the local sensor data and pushes it to the internet. Simultaneously, it reads remote commands from the Anedya cloud and routes them back down into the Bluetooth network to control the Relay.

III. Applications

Because this system is highly scalable and uses multi-hop technology to save Wi-Fi bandwidth and reach distant areas, it can be easily adapted to solve major problems in various real-world situations. Below are four key applications where this project can be directly implemented:

A. Smart Home Automation and Campus Management

I can use this system to control dozens of smart lights, fans, and plugs across a large house or university campus. Usually, connecting dozens of devices directly to a home Wi-Fi router causes the network to crash. By using my BLE mesh setup, only the single gateway device connects to the router. The mesh network easily reaches Wi-Fi blind spots like basements, attics, or backyards by securely bouncing the command signals from room to room.

B. Industrial IoT (IIoT) and Factory Safety

In large factories, normal Wi-Fi signals get blocked by heavy metal machinery and thick concrete walls. I can use this mesh system to constantly monitor the temperature of critical, expensive motors on the factory floor. If a machine begins to overheat, the sensor sends a warning through the mesh to the cloud. Using the cloud dashboard, a manager can instantly send a command to the Relay Node to cut off the machine's high-voltage power supply, preventing electrical fires and expensive damage.

C. Precision Agriculture and Smart Farming

Farms cover huge areas of land where Wi-Fi is impossible, and buying a separate cellular data plan for every single soil sensor is far too expensive. I can place these BLE mesh sensors across acres of crops. The sensors pass the humidity and temperature data from plant to plant until the data reaches the main gateway located at the farmhouse. The relay feature can then be used to automatically open large water valves for irrigation based on cloud commands.

D. Smart Commercial Buildings and Energy Management

Large office buildings waste a massive amount of electricity cooling and lighting empty rooms. I can deploy this system to monitor the climate in different office zones. Based on an automated schedule set in the Anedya cloud, the gateway can send mesh commands to the relay nodes to automatically turn off lights, air conditioning units, and non-essential loads in empty meeting rooms after working hours, leading to significant financial savings.

IV. Working of the Firmware Code

The project relies on a mini operating system called FreeRTOS, which allows the ESP32 microcontrollers to handle multiple tasks at the exact same time without freezing. Here is how the code works for each of the three devices:

1. The Sensor Node (Data Collector): This node is programmed to be a simple, energy-efficient messenger. It uses a built-in hardware timer to wake up every 10 seconds. When it wakes up, it talks to the physical DHT22 sensor to read the current temperature and humidity. Instead of sending a massive data packet, the code squishes these numbers into a tiny, lightweight text string (like `"T:33.4,H:61.7"`). It then broadcasts this tiny string into the Bluetooth Mesh, aiming specifically for the Gateway's address, and goes back to waiting.

2. The Gateway Node (The Network Boss): This is the hardest-working device in the system. Once it successfully connects to both the local Wi-Fi and the Bluetooth Mesh, it runs two endless loops (FreeRTOS tasks) simultaneously in the background:

- *Task 1 (The Uploader):* It constantly listens for the tiny text strings broadcasted by the Sensor Node. When it hears one, it carefully unpacks the string back into numbers, formats them into a neat JSON package, and securely uploads them to the Anedya Cloud.
- *Task 2 (The Downloader):* At the exact same time, every 5 seconds, it reaches out to the Anedya Platform to check for user commands. If it finds a new command (like "ON"), it immediately fires a Bluetooth signal to the Relay Node.

3. The Relay Node (The Action Taker): This node's code is strictly event-driven. It sits quietly and listens to the Bluetooth Mesh. When it receives a verified "ON" or "OFF" command from the Gateway, it triggers a callback function. However, physical relay modules are often wired as "Active-Low" (meaning a 0-volt signal turns the switch ON, and a 5-volt signal turns it OFF). To prevent user confusion, my code automatically flips the logic. When the cloud says "ON" (1), the code outputs a '0' so the physical hardware turns on exactly as expected.

V. System Workflow and Implementation

To make the data flow smoothly in both directions and ensure fault tolerance, I divided the project into four main steps.

A. Phase 1: nRF App Setup and Provisioning

Before the devices can talk, they need to be introduced to each other securely. I used the nRF Mesh phone application to establish the network layout. The setup process was carried out as follows:

1) **Scanning and Provisioning:** I used the app's scanner to detect the unprovisioned ESP32 boards broadcasting their UUIDs. I successfully provisioned them into the mesh network.
2) **Addressing:** During provisioning, I assigned specific Unicast Addresses to ensure precise point-to-point routing: the Gateway received `0x0002`, the Sensor received `0x0004`, and the Relay received `0x0006`.
3) **AppKey Binding:** To ensure the network was completely secure from hackers, I bound a shared Application Key (AppKey) to all the specific device models, encrypting the wireless payloads.
4) **Publication and Subscription Configuration:** Finally, I configured the internal routing paths. I set the Sensor Node to automatically publish its temperature data to the Gateway's address, and configured the Relay Node to listen (subscribe) to commands sent from the Gateway.

B. Phase 2: Sensor to Cloud (Uplink)

This phase handles getting physical data to the internet dashboard.

1) **Reading Data:** Every 10 seconds, the Sensor Node reads the DHT22 sensor.
2) **Sending via Bluetooth:** To save battery, it squishes the data into a tiny text message (like `"T:33.4,H:61.7"` and sends it straight to the Gateway over Bluetooth.
3) **Waiting in Line:** The Gateway catches the text, reads the numbers, and puts them into a waiting line (called a FreeRTOS Queue). This ensures the Gateway doesn't get overwhelmed if too many messages arrive at once.
4) **Uploading:** A background task looks at the waiting line and securely uploads the data to the Anedya Cloud using a standard web request (HTTP POST).

C. Phase 3: Cloud to Relay (Downlink)

This phase handles sending a command from the user's phone or computer to the physical relay switch.

1) **Asking the Cloud:** Every 5 seconds, the Gateway sends a web request to the Anedya Cloud asking, "Are there any new commands for me?"
2) **Reading the Answer:** The cloud replies with a chunk of data (JSON). The Gateway uses a tool called `cJSON` to easily search through the data and find the command name, such as `"ON"` or `"OFF"`.

3) **Checking Memory:** The Gateway checks the command's unique ID to make sure it hasn't already executed this exact command before.
4) **Sending the Signal:** If it is a new command, the Gateway creates a simple 2-byte Bluetooth message and shouts it into the mesh network. The Relay hears it, sees the "ON" signal, and clicks the physical hardware switch.

D. Phase 4: The Communication Rules (Protocols)

I used two different "languages" (protocols) for this project:

- **Local Language (BLE Mesh):** For devices talking locally, I used Bluetooth Mesh. I used a custom model for the sensor data to keep the message short, and a standard official Bluetooth model (Generic OnOff) to control the relay reliably via multi-hop routing.
- **Internet Language (HTTP):** For talking to the cloud, I used HTTP requests. I chose to ask the cloud for updates every 5 seconds (polling) rather than keeping a permanent connection open (like MQTT) because it saves valuable memory on the ESP32.

VI. Problems Faced and Solutions

Building this system required me to fix several hidden bugs and network issues.

A. Wi-Fi Blocking the Internet

Problem: The Gateway successfully connected to the Wi-Fi router, but the terminal printed red DNS errors saying it couldn't reach the Anedya Cloud. University and public Wi-Fi networks often block custom electronics from accessing the internet.
Solution: I bypassed the restrictive Wi-Fi by connecting the Gateway to a personal mobile phone hotspot, which has no firewalls.

B. Missing Bluetooth Code

Problem: When building the code, the compiler threw an error saying 'undefined reference'. To save memory, the ESP32's programming tool automatically deletes code it thinks you aren't using.
Solution: I opened the system settings menu ('menuconfig') and forced the system to include the "Generic OnOff Client" Bluetooth libraries.

C. Message Size Limits

Problem: The Gateway received the cloud command, but threw an error ('Too small publication msg size') when trying to send it over Bluetooth. The default memory box for sending Bluetooth messages was only 3 bytes, but my command was 4 bytes.
Solution: I changed a single line of code to increase the publisher memory box size to 20 bytes, giving the message plenty of room to send.

D. Garbage Data (Structure Padding)

Problem: The Gateway sent the message, but the Relay ignored it. The ESP32's brain was automatically adding empty "garbage" bytes (called padding) into the data structure to make it fit its 32-bit processing style. This corrupted the message.
Solution: I stopped using complex data structures and instead packed the command into a pure, simple 2-byte raw array. This sent a mathematically perfect signal that the Relay understood.

E. Reading Messy Cloud Data

Problem: The cloud API sometimes adds invisible spaces or changes the order of the data it sends back. Because the original code was just searching for exact text matches, it missed the commands.
Solution: I upgraded the code to use `cJSON`. This acts like a smart reader that looks for exact data labels (like searching for the tag "commandId") instead of relying on plain text searching.

VII. Simulation and System Results

The following figures demonstrate the successful bi-directional communication between the Anedya Cloud Platform, the local BLE Mesh network, and the physical hardware setup.

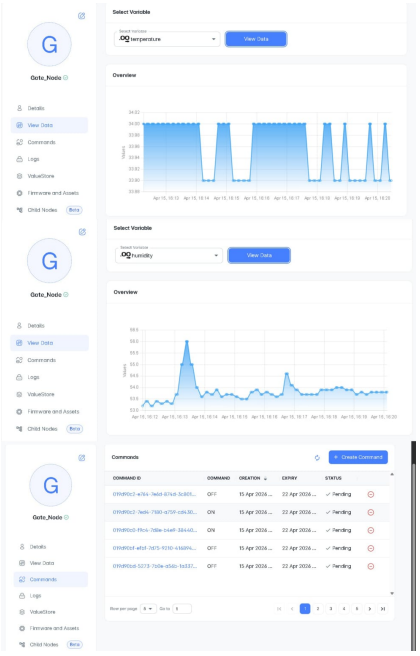

Fig. 1. The Anedya Cloud dashboard displaying the real-time temperature telemetry data and successfully sent user commands.

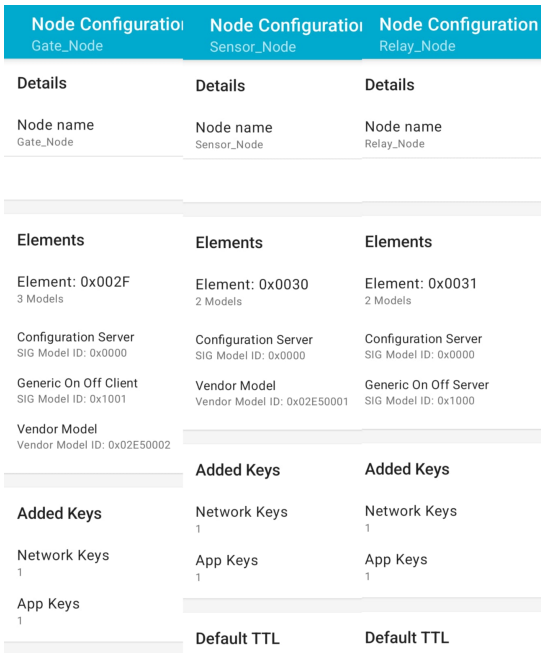

Fig. 2. The nRF Mesh phone app interface showing how I securely linked the devices, bound the AppKeys, and configured the routing.

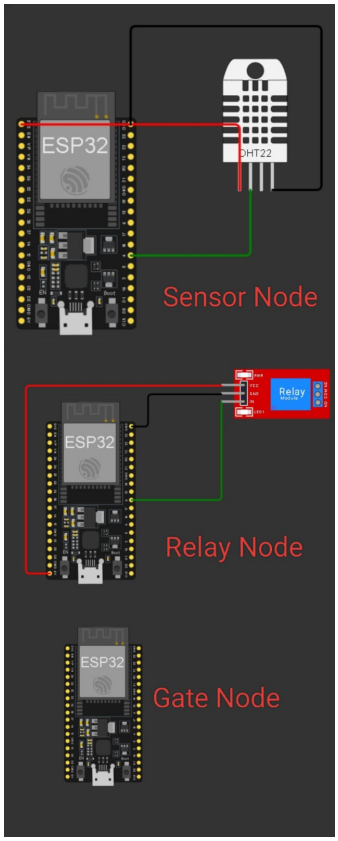

Fig. 3. The hardware circuit diagram illustrating the physical connections between the ESP32 microcontrollers, the DHT22 sensor, and the 5V relay module.

VIII. Conclusion

This project successfully proves that I can connect a local, low-power Bluetooth network to the internet. By fixing several memory bugs, internet blocks, and code formatting issues, I built a highly reliable, two-way bridge. This system allows a user to view sensor data from anywhere in the world and instantly control physical hardware, all while using just a single Wi-Fi connection for the entire network.